

Webcam Filter App

System Architecture and End-to-End Delivery Guide

Project: A local desktop application that captures live webcam video and applies simple visual effects such as shapes, text, and grayscale.

Purpose of this guide: Help students understand how to shape, design, build, test, release, and maintain a small application. The guide treats the app as a product someone owns, not only as a coding exercise.

1. Product scope and assumptions

The first release is a Python desktop app using OpenCV. Video is processed on the user's device and is not recorded, uploaded, or sent to a server. The app has one active camera, one preview window, a small set of effects, and keyboard controls. This keeps the system understandable while still teaching production habits.

- Primary user: a student or user who wants to preview a live webcam with a visual effect.
- Initial effects: original video, grayscale, a simple shape overlay, and text overlay.
- Controls: documented keyboard shortcuts to change effect and quit.
- Target operating systems and supported camera models must be chosen and tested by the project team before release.
- No account, cloud service, database, video recording, analytics, or network connection is required for version 1.

Out of scope for version 1

Face recognition, identity inference, cloud processing, video storage, multi-camera selection, and user accounts. Add such features only after a separate requirements, privacy, and security review.

2. User needs and requirements

Functional requirements

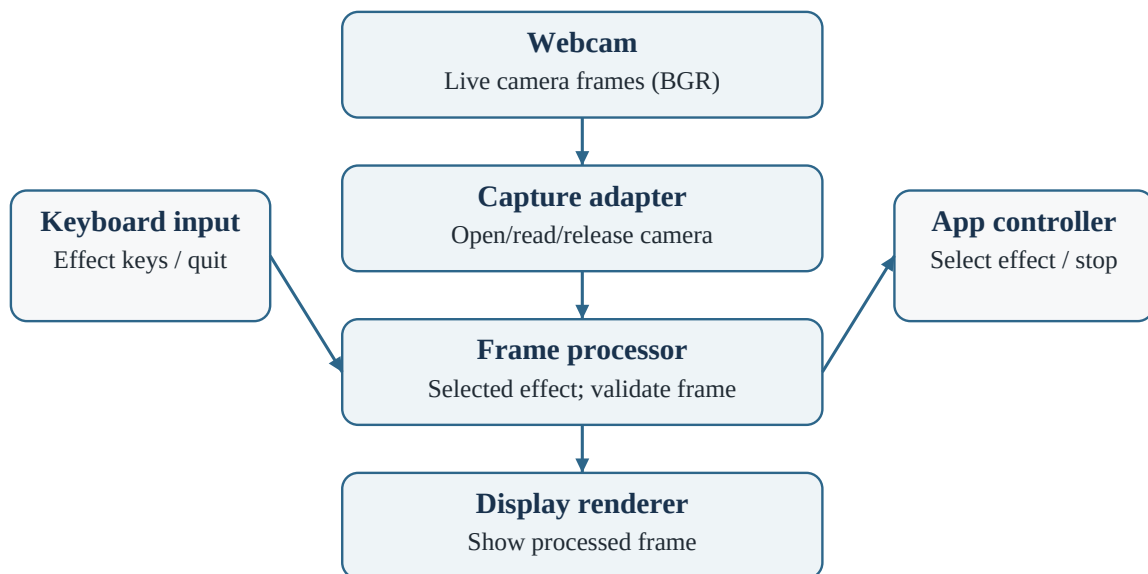
- FR-1: The app opens the selected/default camera and shows a live preview.
- FR-2: The user can switch between the supported effects using visible or documented keyboard shortcuts.
- FR-3: The user can close the app with a quit shortcut or the window close control.
- FR-4: The app reports a useful message if camera access or frame capture fails.
- FR-5: On normal exit and handled errors, the app releases camera resources and closes its window.

Quality requirements

- **Responsiveness:** process and display frames continuously without avoidable work in the loop. Agree and test an acceptable experience on the actual target hardware; do not promise a frame rate before measuring.
- **Reliability:** handle a missing, busy, or disconnected camera without an uncontrolled crash.
- **Privacy:** process frames in memory only; do not save or transmit them.
- **Maintainability:** keep camera I/O, effects, controls, and application lifecycle responsibilities distinct.
- **Usability:** show the user how to change effects and exit; messages should be understandable.

3. System architecture

The design is a single-process, local application. A frame moves from the camera through capture and effect processing to the preview window. Keyboard input changes the selected effect or requests shutdown. There is no backend or persistent application data in version 1.



On exit or failure: release camera and close the display window.

Figure 1. Local processing architecture for the first release.

Component responsibilities

Component	Responsibility
Application controller	Owens startup, main loop, selected effect, keyboard actions, errors, and shutdown order.
Camera adapter	Opens the camera, reads frames, exposes camera status, and releases the device.
Frame processor	Applies the selected effect to a frame and returns a displayable frame. Contains no camera or window-management code.
Display and input adapter	Shows frames and reads user key/window actions through OpenCV.
Configuration	Holds simple settings such as camera index and supported effect mapping; defaults are safe and documented.
Logging	Records useful lifecycle/error events without storing frames, credentials, or unnecessary personal data.

4. Runtime flow and design contracts

Startup, frame loop, and shutdown

Startup: initialize the display, attempt to open the camera, verify that it opened, then enter the main loop. If the camera cannot open, show a clear error and stop cleanly.

Frame loop: read one frame; verify the read succeeded and the frame is valid; apply the currently selected effect; display the output; poll controls; then repeat. The loop should have one clear exit condition.

Shutdown: use a cleanup path that releases the camera and destroys OpenCV windows. Cleanup should run whether the user quits normally or a handled error occurs.

Frame processing contract

Treat a frame as an image array supplied by the capture adapter. The processor accepts a frame and an effect choice, validates required inputs, and returns a frame of displayable dimensions and channel layout. Keep the original frame unchanged when practical; copy it before drawing overlays if the caller needs the original. Effects should be small functions that can be tested without opening a camera.

Failure behavior

Condition	Expected behavior
Camera does not open	Show a concise message, record a non-sensitive diagnostic, exit cleanly.
Frame read fails	Do not pass an invalid frame to processing; stop or report the failure and clean up.
Unsupported effect key	Keep the current effect and continue; optionally show supported controls.
Window closed / quit key	Leave the loop and release resources.
Unexpected exception	Run cleanup, show a safe user-facing error, and retain diagnostic details without image data.

5. User interface and accessibility

- Use a single preview window with a short, readable control hint; keep overlays away from the most important portion of the image.
- Use high contrast and do not communicate effect state by color alone. Include a text label or other distinct cue when practical.
- Use simple, consistent shortcuts and document them beside the app or in a help view. Avoid trapping the user in a window with no obvious exit.
- Ensure window-close actions and keyboard quit both trigger the same cleanup flow.
- If controls expand beyond a few shortcuts, reassess whether an OpenCV window is sufficient and select a more suitable UI framework as a deliberate later design change.

6. Privacy and security

Webcam images can contain sensitive personal information. The default design should minimize collection: frames remain in process memory and are neither saved nor transmitted. A visible preview indicates when the camera is active, and closing the app releases the device.

- Request only camera access needed by the operating system; explain to users why access is required.
- Do not add recording, screenshots, uploads, analytics, or crash attachments containing frames without explicit product requirements, consent, and safeguards.

- Do not log image arrays or frame content. Keep logs limited to operational facts such as startup, camera-open failure, and shutdown.
- Document third-party dependencies and update them deliberately. Pin tested versions and review dependency advisories through the team's normal maintenance process.
- If packaging or installation changes operating-system permissions, test and document the user experience on every supported platform.

7. Code organization and configuration

A small package layout makes responsibilities visible while keeping the first release approachable. Students can begin with fewer files, then split modules when each boundary is understood.

```
webcam-filter-app/  
■■■ README.md  
■■■ pyproject.toml (or requirements files)  
■■■ src/webcam_filter_app/  
■ ■■■ main.py  
■ ■■■ app.py  
■ ■■■ camera.py  
■ ■■■ effects.py  
■ ■■■ display.py  
■ ■■■ config.py  
■■■ tests/  
■■■ test_effects.py  
■■■ test_app_flow.py
```

Module guidance: main.py is the entry point; app.py coordinates the lifecycle; camera.py wraps VideoCapture; effects.py contains pure frame transformations; display.py wraps window and keyboard calls; config.py holds validated defaults. Avoid introducing a database, service layer, or complex plugin framework without a need.

Configuration: use safe defaults and validate values at startup. For a beginner release, a documented default camera index and fixed effect shortcuts may be enough. If configuration becomes user-editable, define where it lives, how invalid values are handled, and how defaults are restored.

8. Development workflow and milestones

Milestone	Deliverable / completion check
1. Understand	Agree on user, supported platform assumptions, feature list, privacy behavior, and acceptance criteria.
2. Design	Review this architecture, sketch UI states, define module responsibilities, and record unresolved decisions.
3. Camera spike	Open and display webcam video; verify failure and shutdown behavior on target machines.
4. Effects	Implement and test grayscale and drawing/text effects using sample frames, then connect them to live capture.
5. Controls and errors	Add shortcuts, window-close handling, friendly errors, and guaranteed resource cleanup.
6. Quality pass	Run automated tests, manual camera checks, usability review, and dependency/security checks.
7. Release	Write setup instructions, pin tested dependencies, package if needed, test installation, and publish a versioned release.
8. Own and improve	Track issues, maintain dependencies, respond to user feedback, and record decisions for future changes.

Working with AI assistance

- Ask an AI assistant to explain a proposed design or generate a small, bounded change; provide the relevant constraints and file context.
- Review generated code line by line, run it, and ask for explanations of unfamiliar OpenCV behavior. Students remain responsible for correctness, privacy, licensing, and maintainability.
- Do not provide real webcam footage, personal data, credentials, or private user information to an AI tool. Use synthetic or non-sensitive test frames.
- Keep changes small: define the task, inspect the diff, run relevant tests, and document accepted decisions.

9. Testing and acceptance

Test strategy

- Unit tests: verify effect functions on small synthetic image arrays, including expected shape/type and unchanged input where required.
- Component tests: use a fake camera or controlled frame source to test success, failed reads, effect selection, and cleanup without relying on a physical webcam.
- Manual integration tests: on each supported platform, verify camera permission, preview, shortcuts, window close, disconnect behavior, and relaunch after exit.
- Release checks: install from the documented instructions in a clean environment and repeat the main user journey.

Version 1 acceptance checklist

- A first-time user can install or run the app using the README.
- The camera preview starts when permission and hardware are available.
- Each documented effect can be selected and behaves as described.
- A missing or unavailable camera produces a clear failure and no invalid-frame crash.
- Both quit mechanisms close the app and release the camera.
- No video is written to disk or sent over the network.
- Tests pass and the release has a version identifier and known platform support.

10. Release and deployment

For a teaching project, “deployment” means giving another user a repeatable way to run the desktop app. Start with a virtual environment and documented installation. Pin the dependency versions that the team tested. If a standalone executable is desired, evaluate a packager such as PyInstaller only after the script-based application works; packaging, camera permissions, and OpenCV window behavior need platform-specific verification.

- Provide a README with prerequisites, setup, run instructions, controls, camera troubleshooting, supported operating systems, and privacy behavior.
- Use version control. Keep generated environments, caches, and local footage out of the repository.
- Tag releases and include known limitations. Do not claim support for untested platforms or hardware.
- Keep a reproducible dependency definition; update and retest it rather than changing packages ad hoc on user machines.

11. Operations, support, and ownership

A desktop app still has operational responsibilities. The owner is responsible for installation guidance, issue triage, dependency updates, compatibility checks, and communicating limitations. Agree who maintains the repository and how users report a problem.

Issue report should include	Do not request by default
App version, operating system, Python/package version, camera model if known, exact steps, and the error message.	Webcam recordings, screenshots containing people, passwords, personal documents, or unrelated personal data.

For an error report, first reproduce with synthetic frames or another non-sensitive test. Ask for additional diagnostic information only when necessary, explain why it is needed, and give users a way to redact identifying details.

12. Decisions to revisit before expanding

- Which operating systems and minimum hardware should version 1 support?
- Should camera selection or resolution become user-configurable?
- Is a graphical control panel justified beyond the simple preview and keyboard interaction?
- Would any proposed recording or sharing feature need new consent flows, storage controls, retention rules, and a security review?
- What maintainers, issue process, and release cadence will the project use?

13. End-to-end student deliverables

By completing the project, students should be able to show more than working code. The project handoff should include:

- A short product brief and prioritized requirements.
- A reviewed component diagram and key design decisions.
- Readable source code with a clear entry point and separated responsibilities.
- Automated tests plus a documented manual test checklist.
- A README and reproducible setup instructions.
- A versioned release or a clear demonstration of how another person can run the app.
- A brief ownership note covering privacy behavior, supported platforms, known limitations, and how issues will be handled.